

AI TOOLING VISIBILITY

Following the AI Audit Trail

A map of what agents log, where to find it, and what it misses

Darwin Salazar
The Cybersecurity Pulse
@darwnsm

monad

$ whoami

role	**Head of Growth,** Monad. Security data pipelines.
before	**Detection engineering,** Datadog **Cloud security,** Accenture **Medical device security,** Johnson&Johnson
specialty	Azure and Microsoft Sentinel KQL kung-fu workshop at DEF CON
writes	The Cybersecurity Pulse on Substack
home	Austin, TX



---

disclosure — This research comes from the work we do at Monad.

AI audit logs in 2026 are where cloud audit logs were in 2018

> Field names change between versions.
>
> Products merge. Cowork and chat are merging; rollout varies by deployment.
>
> Users adopt fast. Security is still figuring out what gets logged.

HANDS UP

Who’s running a coding agent? Keep it up if you’re collecting logs. Keep them up if your rolling your own detections for it.

What an agent can touch

Code
Reads and edits the repo. Commits use the developer's git identity; AI attribution can be turned off.

Local files
Anything the user can read. ~/.aws, ~/.ssh, .env, kubeconfig.

Shell
Runs commands as the user, inside the permission mode and sandbox.

APIs and MCP servers
Calls services with the tokens it was given. Data can leave through a tool call.

Cloud
Reads data and changes infrastructure using the identity tied to its credentials.

SaaS
Reads, exports, shares, deletes. A bulk export is one tool call.

Attribution is messy, and agents inherited the traditional security model.

Four questions, four kinds of evidence

Who changed access, keys, or policy?	Platform audit records Vendor audit APIs.
What did the agent try to do, and who or what approved it?	Runtime events Usually over OpenTelemetry
What instructions or content were involved?	Transcripts and content capture Prompt text, tool inputs, responses. Usually off.
What actually happened to the system or data?	Evidence from outside the agent Endpoint, network, destination logs. Not in the agent's telemetry.

Who gives you what, and what it takes to get it

Source	Gives you	How	Plan or setting	Enabled by	Command, prompt text
Claude Code	runtime events	push	any install	developer, or managed settings	off by default
Claude Cowork	runtime events	push	Team, Enterprise	org admin	Version/config dependent
OpenAI Codex	runtime events	push	any, opt-in	developer	prompts redacted
Cursor	audit + runtime	poll + push	Enterprise	Cursor admin	separate opt-in
GitHub Copilot	audit + runtime	poll + push	Varies by feed	Org / enterprise / client	Client OTel: off by default.
Anthropic Activity Feed	audit	poll	Compliance API on	compliance key	none
Anthropic transcripts	content	poll	Enterprise, Compliance on	compliance access key	the content itself
OpenAI API Platform	audit	poll	owner must enable	org owner	none
Gemini for Workspace	feature usage	poll	Workspace edition	Workspace admin	none

Roles and keys: A9. Field by field: A1, A2.

Cloud audit logs then, agent telemetry now

The same questions still help. Watch where the comparison breaks down.

	Azure Activity Log	AWS CloudTrail	Claude Code
When	eventTimestamp	eventTime	event.timestamp
Who	caller	userIdentity	user.email, when available
What	operationName	eventName	tool_name
With what	properties	requestParameters	tool_parameters, only with OTEL_LOG_TOOL_DETAILS=1
Did it work	status: Started, then Succeeded or Failed	one event; errorCode on failure	tool_decision, then tool_result with success
Tie it together	correlationId groups the operation	requestID; no pair to join	tool_use_id joins the pair
Session	no conversation	sessionContext is a credential session, not a conversation	session.id is a conversation; prompt.id, one turn in it

Coverage. Cloud: control plane on, data plane opt-in. AI: audit logs gated by plan; runtime events off until enabled.

History. Cloud and AI audit APIs keep a retention window. Runtime telemetry: no backfill.

Who writes it. Audit logs: the platform. Runtime events: the client on the developer's machine.

One tool_result, before and after flattening

AS RECEIVED, OTLP/JSON

```
{"resourceLogs": [{"resource": {…},
  "scopeLogs": [{"logRecords": [{
    "timeUnixNano": "1792042683513000000",
    "body": {"stringValue": "claude_code.tool_result"},
    "attributes": [
      {"key": "session.id",
       "value": {"stringValue": "fb7c03fa…"}},
      {"key": "tool_use_id",
       "value": {"stringValue": "toolu_01Mzam…"}},
      {"key": "decision_source",
       "value": {"stringValue": "user_temporary"}},
      {"key": "success",
       "value": {"stringValue": "true"}},
      {"key": "tool_parameters",
       "value": {"stringValue":
        "{\"bash_command\":\"cp\",\"full_command\":…"}},
     … 6 more ]}]}]}]}
```

OTLP/JSON shape, run 1 values. Every value is a string.
Open source: monad-inc/community-transformations

AFTER SPLIT AND FLATTEN

1	event.timestamp	"2026-10-15T05:38:03.513Z"
1	user.email	"antony.santos@tcp.com"
2	session.id	"fb7c03fa…"
2	prompt.id	"323e50e6…"
2	tool_use_id	"toolu_01Mzam…"
3	tool_name	"Bash"
3	tool_parameters	parsed into an object
	bash_command	"cp"
	full_command	"cp ./demo-secrets/FAKE_aws_credentials ./backup/FAKE_aws_credentials.bak"
4	decision_type	"accept"
4	decision_source	"user_temporary"
5	success	"true"

Run 1, Oct 15, 2026. Subset, reordered.

1 **Event time and user.**

2 **Session, turn, and tool call.**

3 **What it ran.**

4 **What allowed it.** A person, this time.

5 **The result reported by the tool.**

What a Claude Code session is

Session

You start claude. Every event gets the same `session.id` until you exit. Resume keeps it; /clear starts a new one.

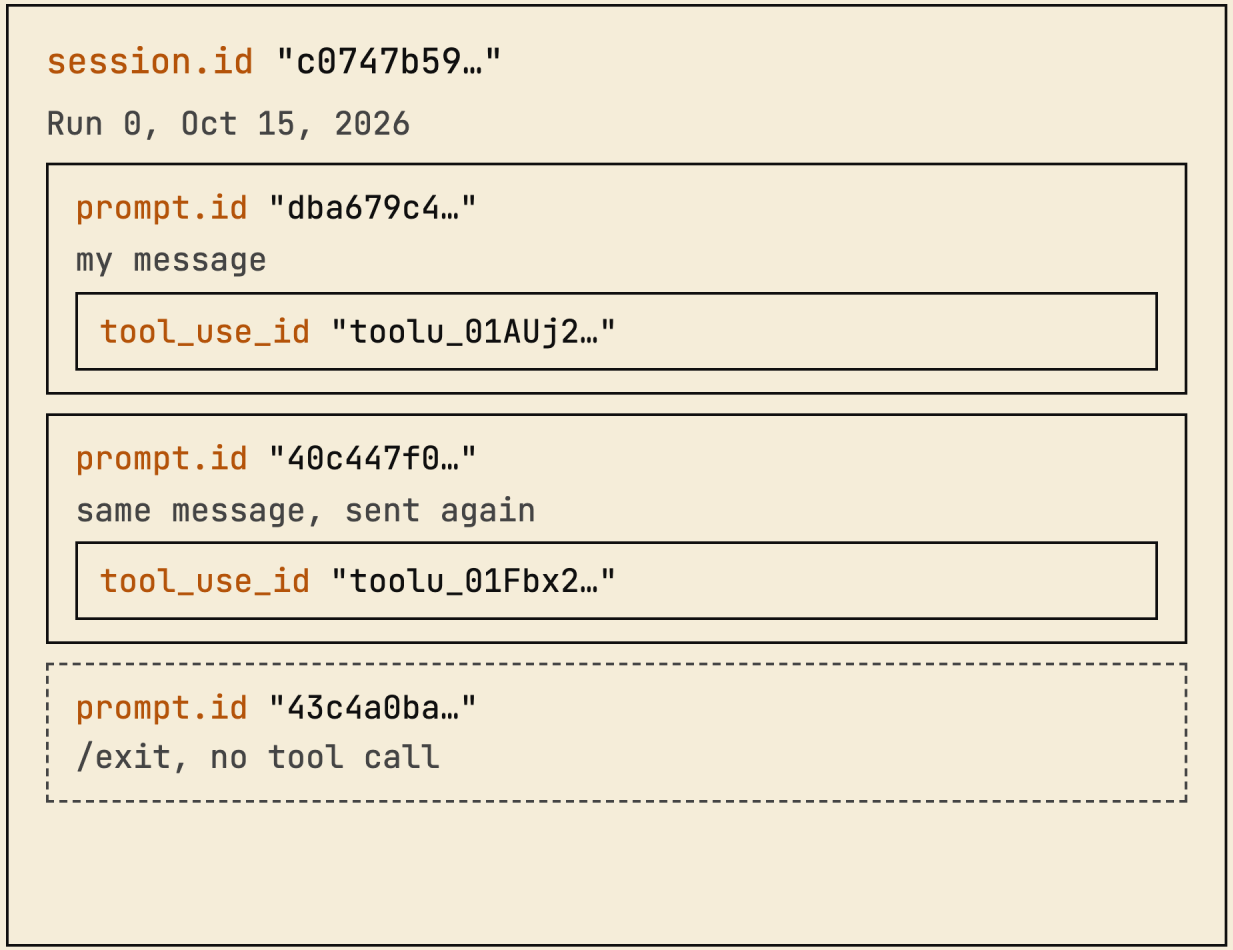
Turn

You send a message. Everything the agent does to answer it shares one `prompt.id`.

Tool call

The agent runs a tool. One `tool_use_id`, two events: `tool_decision`, then `tool_result` if it ran. Join on the ID, not on time.

The boxes show shared IDs, not nesting. Every event is flat and carries all three.



What’s in the record, and how it reaches your SIEM

WHAT’S IN IT

~/.claude/settings.json (home folder, every project)

```
"env": {
  "CLAUDE_CODE_ENABLE_TELEMETRY": "1",
  "OTEL_LOGS_EXPORTER": "otlp",
  "OTEL_EXPORTER_OTLP_PROTOCOL": "http/json",
  "OTEL_EXPORTER_OTLP_ENDPOINT": "http://127.0.0.1:4318",
  "OTEL_LOG_TOOL_DETAILS": "1"
}
```

OTEL_LOG_TOOL_DETAILS=1 adds full_command.
OTEL_LOG_USER_PROMPTS=1 would add prompt text; I left it off, so prompts show as "<REDACTED>".

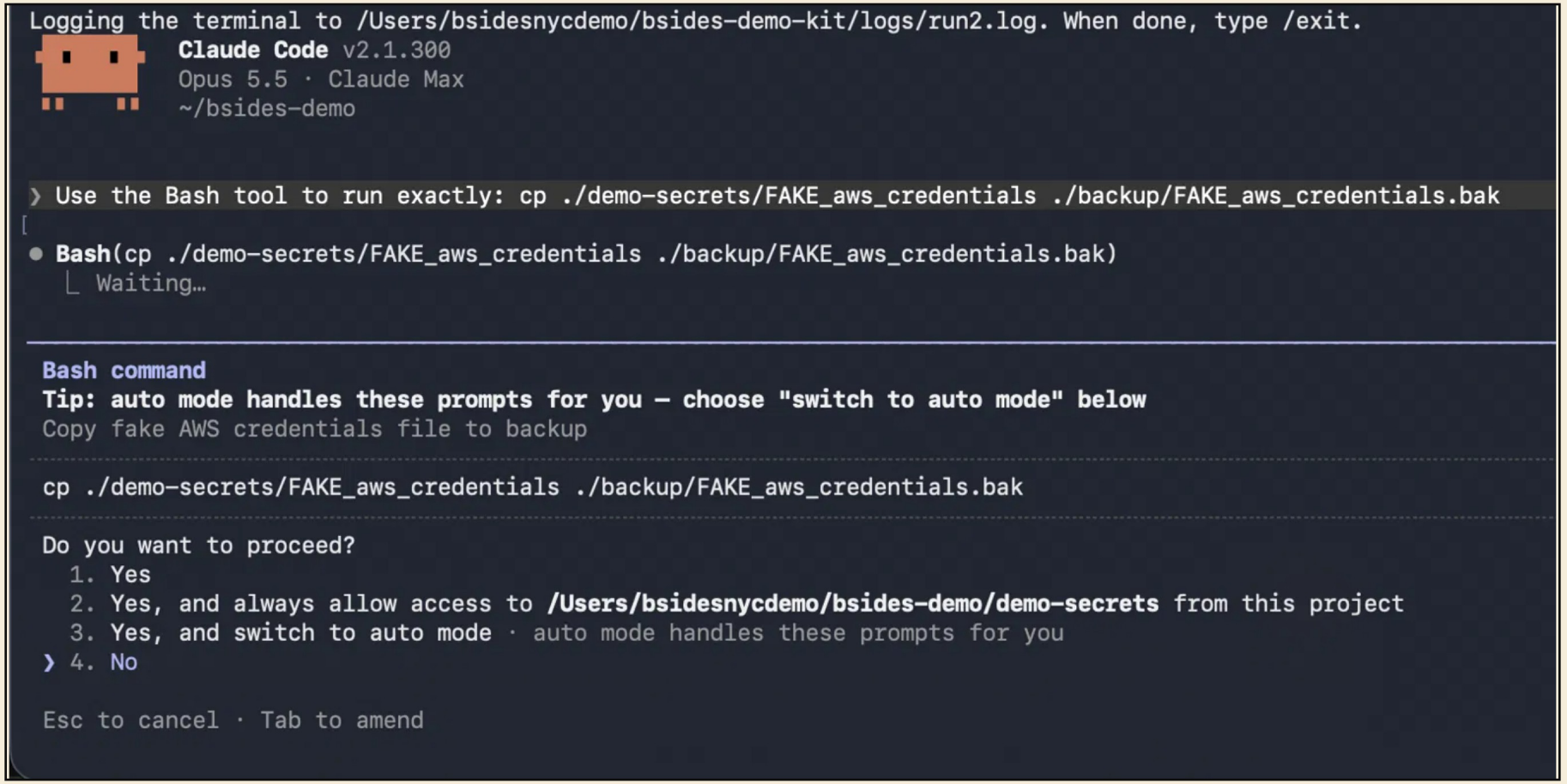
HOW IT GETS OUT

Push: Claude Code’s only option
The client sends events to your collector as they happen. No backfill.

Poll or stream: vendor audit APIs
Audit logs from Anthropic, OpenAI, GitHub, Cursor. Pull history within retention. No Claude Code tool events.

In between
A pipeline (Monad, or your own) splits, flattens, normalizes, redacts. It can’t recover what was never recorded.

The permission prompt



Manual: asks before edits, shell commands, network. Runs 1 and 2.

Auto: a classifier model reviews actions instead of you. Run 0.

Who approves the tool call?

Who answers	source
Yes	"user_temporary"
Yes, always allow	"user_permanent"
No	"user_reject"
No prompt in run 0.	"config"

2.1.300 groups several permission paths under config. 2.1.301 adds classifier_verdict and permission_mode.

Manual approval

RUN 1

tool_decision

event.timestamp	"2026-10-15T05:38:03.408Z"
session.id	"fb7c03fa…"
tool_use_id	"toolu_01Mzam…"
tool_name	"Bash"
decision	"accept"
source	"user_temporary"
full_command	parsed from tool_parameters

```
cp ./demo-secrets/FAKE_aws_credentials
./backup/FAKE_aws_credentials.bak
```

tool_result

event.timestamp	"2026-10-15T05:38:03.513Z"
tool_use_id	"toolu_01Mzam…"
decision_type	"accept"
decision_source	"user_temporary"
success	"true"

LOOK AT THE RECORD

Does this record show that a person approved the call?

Yes.

source

decision_source

I clicked Yes once. user_temporary appears on both events; tool_use_id ties them together.

Manual rejection

RUN 2

```
== tool_decision ================================
  event.timestamp  "2026-10-15T05:42:30.475Z"  (stringValue)
  session.id       "fbdb4fba-8ad1-4208-91c4-9aad32291956"  (stringValue)
  prompt.id        "ff781421-085d-42d6-a28d-cf970717a7d0"  (stringValue)
  tool_name        "Bash"  (stringValue)
  tool_use_id      "toolu_019LXFLHyA8mBeQcnQNAoAke"  (stringValue)
  decision         "reject"  (stringValue)
  source           "user_reject"  (stringValue)
  tool_parameters  "{\"bash_command\":\"cp\",\"full_command\":\"cp ./demo-s..."  (stringValue)
    -> full_command cp ./demo-secrets/FAKE_aws_credentials ./backup/FAKE_aws_credentials.bak
    -> bash_command cp
    -> description  Copy fake AWS credentials file to backup
```

```
Receiver stopped. 15 event(s): api_request 2, assistant_response 1, managed_settings_resolved 1, mcp_server_connection 4
, plugin_loaded 4, tool_decision 1, user_prompt 2
```

The rejected call is still in `tool_decision`

`decision` "reject", `source` "user_reject".

`tool_parameters` still holds the command

It never ran.

No tool_result.

There are 15 events in this run and no tool_result.
A detection that only looks at results misses the rejected attempt.

Automatic approval

RUN 0 BESIDE RUN 1

Run 1, Manual. I clicked Yes.

tool_decision

event.timestamp	"2026-10-15T05:38:03.408Z"
tool_use_id	"toolu_01Mzam…"
decision	"accept"
source	"user_temporary"

tool_result

tool_use_id	"toolu_01Mzam…"
decision_source	"user_temporary"
success	"true"

Run 0, auto. No prompt.

tool_decision

event.timestamp	"2026-10-15T05:44:14.672Z"
tool_use_id	"toolu_01AUj2…"
decision	"accept"
source	"config"

tool_result

tool_use_id	"toolu_01AUj2…"
decision_source	"config"
success	"true"

Both runs used the same command and reported success. Only one shows that I clicked Yes.

In this auto run, I wasn't prompted. These captures use 2.1.300. 2.1.301 adds classifier_verdict (what the auto-mode classifier decided) and permission_mode (which mode was on), so you can tell the classifier from a rule.

Where to start

1 **Find out what’s running**
List the agents, where they run, and your plans. Use slide 6 as the checklist.

2 **Turn it on, keep it on**
Start client telemetry first; it doesn’t backfill. Deploy it through managed settings so it stays on.

3 **Decide on content capture**
Without OTEL_LOG_TOOL_DETAILS=1, you won’t see the command. Protect those logs like the source data.

4 **Go try it yourself**
Point Claude Code’s OTel at a local collector (setup in A4). Click Yes, click No, try auto mode.

5 **Learn how the tool behaves before you write the detection**
Every agent logs differently, with its own quirks. To know what 'anamolous' looks like, you must know what 'normal' looks like at the log level

If you want to go deeper

AI Agent Artifact Catalog
depalmar.github.io/ai-dfir-toolkit
What agents, model runtimes, and MCP components leave on an endpoint, and the order to collect it.

MITRE ATLAS
atlas.mitre.org
Agentic techniques since early 2026, e.g. AML.T0110 AI Agent Tool Poisoning. No ATT&CK for agents.

OWASP Top 10 for Agentic Applications
genai.owasp.org, Dec 2025
ASI01 Agent Goal Hijack through ASI10 Rogue Agents.

Claude Code monitoring docs
code.claude.com/docs/en/monitoring-usage
Every field name in this talk. The names move; re-read each release.

Monad community transformations
github.com/monad-inc/community-transformations
The split-and-flatten from slide 8, plus an OCSF path. Open source.

Detection engineering for Claude Code, parts 1 and 2
monad.com/blog, Mar 2026
Five starter rules against the flattened record. Where this talk started.

Thank you

Questions?

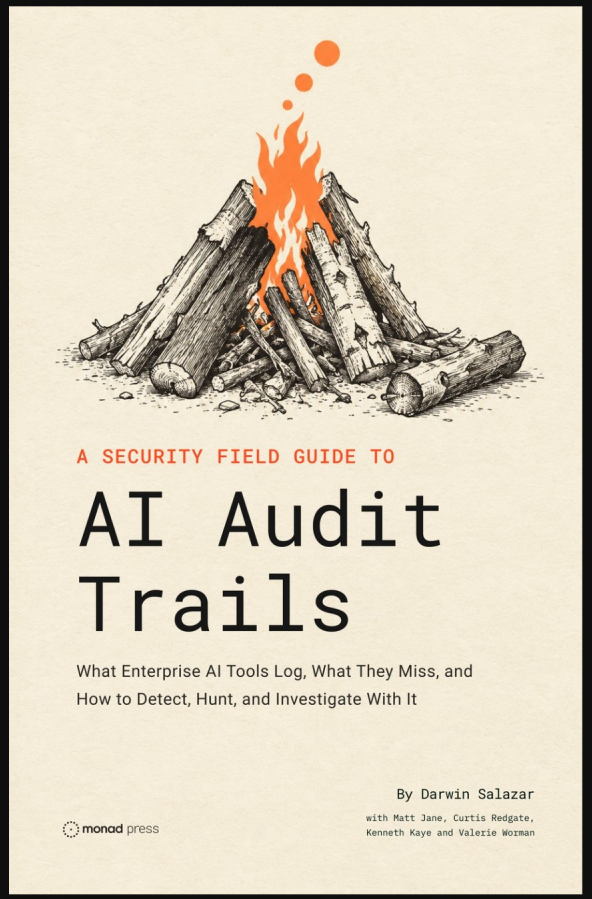
---

slides github.com/darwinsec

find me @darwnsm

monad

The Cybersecurity Pulse



Free copies today!

BACKUP SLIDES

Appendix

A1	Source comparison: agents and developer tools	**A2**	Source comparison: enterprise AI platforms
A3	Claude Code telemetry settings	**A4**	Reproduce this on your laptop: setup
A5	Reproduce this on your laptop: the three runs	**A6**	Other sources for the demo
A7	Source notes	**A8**	References
A9	What you need to collect each source	**A10**	Starting hunts, not alerts

REVIEWED OCT 1 TO 7, 2026

Source comparison: agents and developer tools

	Claude Code	Claude Cowork	OpenAI Codex
Evidence	Runtime events, metrics; traces in beta	Runtime events	Runtime events, metrics, traces
Reaches you	Client push (OTLP)	Push (OTLP over HTTP)	Push; [otel] in user-level ~/.codex/config.toml
Who can get it	Any install; user or managed settings, not repo settings	Team and Enterprise. Desktop 1.1.4173+ local, 1.22209.3+ cloud	Opt-in. Project-local otel settings are ignored
Content	Prompts and tool details: separate opt-ins, both off	Version/config dependent; check otlpContentCapture.	Prompts redacted unless enabled; results may include output
Watch for	tool_parameters is a JSON string	Local and cloud sessions differ; verify both	--yolo and similar flags need endpoint logs

Cowork OTel settings rechecked Oct 10. Codex details not rechecked.

REVIEWED OCT 1 TO 7, 2026

Source comparison: agents and developer tools

	Cursor	GitHub Copilot
Evidence	Audit (control plane); Enterprise OTel export	Audit; enterprise usage records; client OTel
Reaches you	Audit: API poll or vendor stream. OTel: sent from Cursor’s side	Audit API or streaming; usage records (48-hour pull window); client push
Who can get it	Enterprise plan	Org audit: Enterprise Cloud, org owner. Usage records: EMU enterprise owner, preview. OTel: client or managed config.
Content	Conversation content is a separate opt-in; Cloud Agents and Grok Bot only	Client OTel content capture off by default
Watch for	team_command is not shell-command telemetry	actor:Copilot covers the last 180 days

Cursor plan and GitHub’s 180-day window rechecked Oct 10.

RECHECKED OCT 10, 2026

Source comparison: enterprise AI platforms

	Anthropic Activity Feed	OpenAI API Platform audit logs	Google Workspace Gemini activity
Evidence	Activity metadata: identity, admin, resource events	Audit: identities, roles, keys, network and org controls	Feature usage: user, time, IP, app, action
Reaches you	Poll GET /v1/compliance/activities	Poll GET /v1/organization/audit_logs with an Admin key	Poll Admin SDK Reports API (gemini_in_workspace_apps)
Who can get it	Compliance API enabled; key with read:compliance_activities	An owner must enable audit logging	Admin with admin.reports.audit.readonly; edition-dependent
Content	None. Transcripts need read:compliance_user_data	None. API call logging is a separate setting	None. Vault covers Gemini app conversations only
Watch for	Not retroactive. Queryable within 1 minute; kept 6 years. Org fields can be null	Tenant events need tenant_only=true; no fixed retention	Available from 2025-06-20; rolling 180-day history

Activity Feed timing and retention, OpenAI enablement, and Gemini availability rechecked Oct 10, 2026.

CHECKED OCT 10, 2026

Claude Code telemetry settings

Setting	What it does	Default
CLAUDE_CODE_ENABLE_TELEMETRY=1	Turns telemetry on (required)	Off
OTEL_LOGS_EXPORTER / OTEL_METRICS_EXPORTER	otlp exports events / metrics	Unset
OTEL_EXPORTER_OTLP_PROTOCOL	grpc, http/json, or http/protobuf	None; must be set
OTEL_EXPORTER_OTLP_ENDPOINT / _HEADERS	Receiver address and authentication	Unset
OTEL_LOG_USER_PROMPTS=1	Prompt text on user_prompt	Off (redacted)
OTEL_LOG_ASSISTANT_RESPONSES	Response text; 0 keeps it redacted	Follows the prompt setting
OTEL_LOG_TOOL_DETAILS=1	tool_parameters and tool_input	Off
OTEL_LOG_RAW_API_BODIES	Full request and response JSON	Off
OTEL_LOG_TOOL_CONTENT=1	Tool output span event; needs tracing	Off
OTEL_LOGS_EXPORT_INTERVAL	Event batch interval	5000 ms
CLAUDE_CODE_OTEL_CONTENT_MAX_LENGTH	Content truncation limit	61,440 UTF-16 units

A repository’s .claude/settings.json cannot set exporter variables. Use user or managed settings.

APPENDIX A4

Reproduce this on your laptop: setup

Machine and workspace

I used Claude Code 2.1.300. Run claude --version and save yours.

Dedicated OS user: no real credentials, no SSH keys

~/bsides-demo/demo-secrets/FAKE_aws_credentials, fake on every line

~/bsides-demo/backup/ empty

~/.claude/settings.json env block, not repo settings

CLAUDE_CODE_ENABLE_TELEMETRY=1

OTEL_LOGS_EXPORTER=otlp

OTEL_EXPORTER_OTLP_PROTOCOL=http/json

OTEL_EXPORTER_OTLP_ENDPOINT=http://127.0.0.1:4318

OTEL_LOG_TOOL_DETAILS=1

OTEL_LOG_USER_PROMPTS unset: prompts stay <REDACTED>

Local telemetry receiver

Any OTLP/HTTP receiver bound to 127.0.0.1:4318 that accepts JSON on /v1/logs

Keep what arrives (as received) and a flattened copy for reading. The kit writes run*.otlp.jsonl and run*.events.json

Keep the original payload. Flattening changes its structure.

Start, prompt, exit

Manual mode: cd ~/bsides-demo && claude --permission-mode default

Auto mode (the built-in default on 2.1.283+): cd ~/bsides-demo && claude

Prompt, exactly: Use the Bash tool to run exactly: cp ./demo-secrets/FAKE_aws_credentials ./backup/FAKE_aws_credentials.bak

/exit after each run. One session per run, so each gets its own session.id

APPENDIX A5

Reproduce this on your laptop: the three runs

Run	Start	You click	tool_decision	tool_result
1	claude --permission-mode default	Yes, once	accept, source user_temporary	emitted; success "true"; decision_source user_temporary
2	claude --permission-mode default	No	reject, source user_reject	not emitted; full_command still on the decision
0	claude (auto mode, the built-in default on 2.1.283+)	No prompt	accept, source config	emitted; success "true"; decision_source config

Join decision to result on tool_use_id. Event counts in the captured runs: 18 (run 1), 15 (run 2), 26 (run 0, two calls).

Approved is tool_decision. Reported is tool_result with success the string "true". Verified is not in these logs: a file-write event supports the write; compare contents separately.

These captures use 2.1.300; 2.1.301 adds classifier_verdict and permission_mode.

Other sources for the demo, and what changes

Compliance transcripts

POLL

- Needs Claude Enterprise, Compliance on, and a Compliance Access Key with read:compliance_user_data; Admin keys can’t read transcripts
- Covers local Claude Code and Cowork sessions; not cloud Code, Console API key, or third-party sessions; ZDR excluded

WHAT YOU GET

You get the request and tool input without client telemetry. Endpoint evidence is still needed.

Compliance Activity Feed

POLL

- Needs a key with read:compliance_activities
- Records account and organization activity, not local commands

WHAT YOU GET

This feed records account and organization activity, not the local copy.

EDR on the demo host

PUSH, VENDOR AGENT

- Needs an endpoint agent on the demo machine
- Shows the cp process, its parent, and the file write

WHAT YOU GET

A matching file-write event would independently support the write.

Source notes

CONFIRMED

- Claude Code event and attribute names, content switches, and defaults (events every 5 s, metrics every 60 s)
- Activity Feed: queryable within 1 minute, kept 6 years, not retroactive; file and artifact names null since Sep 24, 2026
- Cursor audit logs and OTel export: Enterprise plan
- GitHub actor:Copilot filter: last 180 days
- Cowork OTel: Team and Enterprise; Desktop version minimums
- Gemini activity: from 2025-06-20, 180-day window

WHAT CHANGED

- Cowork setup path: Help Center says Organization settings > Cowork; the book says Admin settings > Cowork
- Monad’s Mar 26 post: Bash commands appear by default. Current docs: tool_parameters needs OTEL_LOG_TOOL_DETAILS=1. The book matches the docs
- Cowork is merging into Claude: announced Sept 16, 2026; Unified Claude beta via desktopHome since Oct 8 (Cowork changelog, read Oct 15). Unified cloud-session telemetry is documented.
- That post lists two transform paths (ocsf:v1.1.0 and utility:v1.0.0) for claude-otlp-flatten
- OpenAI audit logs: no fixed retention; can’t be disabled from settings once on

APPENDIX A8

References

The book	Salazar, Jane, Kaye, Redgate, Worman. A Security Field Guide to AI Audit Trails. Monad, October 2026 edition
Claude Code	code.claude.com/docs/en/monitoring-usage
Claude Cowork	support.claude.com/en/articles/14477985, claude.com/docs/cowork/monitoring
Anthropic Compliance	platform.claude.com/docs/en/manage-claude/compliance-activity-feed, …/compliance-sessions, …/compliance-api-access
OpenAI Codex	learn.chatgpt.com/docs/config-file/config-advanced, …/config-reference
Cursor	cursor.com/docs/enterprise/compliance-and-monitoring, …/opentelemetry-export
GitHub Copilot	docs.github.com/en/copilot/reference/agentic-audit-log-events, docs.github.com/en/enterprise-cloud@latest/copilot/concepts/enterprise/opentelemetry
OpenAI API Platform	help.openai.com/en/articles/9687866, developers.openai.com/api/reference (audit logs)
Google Workspace	developers.google.com/workspace/admin/reports/v1/appendix/activity/gemini-in-workspace-apps
Monad	monad.com/blog/detection-engineering-for-claude-code-part-1, …-part-2, github.com/monad-inc/community-transformations

Full reference list: the book, pp. 103 to 109.

APPENDIX A9

What you need to collect each source

Feed	Plan or setting required	Role or permission required
Claude Code runtime OTel	None. Any install.	The developer, or your org via managed settings
Claude Cowork runtime OTel	Team and Enterprise; Desktop version minimums	Org admin, Admin settings > Cowork
Anthropic Activity Feed	Compliance API enabled	Key with read:compliance_activities
Anthropic session transcripts	Claude Enterprise, Compliance on	Compliance Access Key with read:compliance_user_data; Admin keys can’t
OpenAI Codex runtime OTel	None. Opt-in in user config.	The developer
OpenAI API Platform audit logs	Must be enabled	Org owner to enable; Admin API key to read
Cursor audit logs, OTel export	Enterprise plan	Cursor admin
GitHub Copilot feeds	Audit: Enterprise Cloud. Usage: EMU, preview. OTel: configured.	Audit: org owner. Usage: enterprise owner. OTel: user or managed settings.
Gemini for Workspace activity	Depends on your Workspace edition	Admin with admin.reports.audit.readonly

Rechecked Oct 10 where A1 and A2 say so.

APPENDIX A10

Starting hunts, not alerts

Sensitive path in the command (hunt)

`full_command` matching .aws/credentials, .ssh, .env, kubeconfig, .azure. Key it off `tool_decision` so rejected calls count.

Path text doesn't prove the file was read or that anything left. It tells you where to look next. Needs OTEL_LOG_TOOL_DETAILS=1.

Accepted decision, no result (collection check)

`tool_decision` accept with no `tool_result` on the same `tool_use_id`.

Could be a long-running action, an interruption, or a collection gap. Rejected calls never have one. Check the pipeline before the person.

Automatic approval against policy (policy check)

`source` "config" on an action your policy says needs a person.

Compare the approval source with the team's policy. On 2.1.301+, also inspect `permission_mode` and `classifier_verdict`.

Silence (collection health)

Per device and `user.email`, last event seen, compared with expected activity.

A quiet client isn't broken by itself. Quiet on a machine that should be busy is worth a look.

Start with commands that touch sensitive paths or approvals that don't match the team's policy. Check what else happened in the same session.

Monad's Mar 26, 2026 post (Jane, Redgate) has queries for the sensitive-path and rejected-command cases, and an unknown-MCP-server rule on `tool_parameters`.mcp_server_name. Check its field names against your pipeline.